

Asenkron EF Core ve Tam CRUD API

Veritabanı işlemlerini async/await ile çalıştır; tam güncelleme, kısmi güncelleme ve silme endpoint'leriyle ürün API'sini tamamla.

Yaklaşık	Odak	Çıktı
210–270 dk	PUT, PATCH, DELETE	Tam CRUD API

Bugünün çalışma kuralı

Önce mevcut GET ve POST işlemlerini async yap. PUT, PATCH ve DELETE endpoint'lerini ancak bu testler geçtikten sonra ekle.

Bitiş ölçütü

- Task ve await akışını EF Core üzerinde kullanmak
- PUT ile ürünün tamamını güncellemek
- PATCH ile yalnızca fiyatı değiştirmek
- DELETE ile ürünü silip 204 döndürmek

1. PUT mu, PATCH mi?

İkisi de güncelleme için kullanılır; fark gönderilen değişikliğin kapsamıdır.

Metot	Amaç	Örnek body
PUT	Kaynağın güncellenebilir bütün alanlarını gönderir.	Name + Price
PATCH	Kaynağın yalnızca belirtilen bölümünü değiştirir.	Sadece Price

PUT örneği

```
PUT /api/products/1

{
  "name": "Mekanik Klavye",
  "price": 1500
}
```

PATCH örneği

```
PATCH /api/products/1/price

{
  "price": 1750
}
```

Kullanacağımız yaklaşım

Gün 12'de PUT ile Name ve Price birlikte değişecek. PATCH örneğinde yalnızca Price değişecek; Name mevcut değerini koruyacak.

CRUD eşleşmesi

```
Create → POST
Read   → GET
Update → PUT veya PATCH
Delete → DELETE
```

2. EF Core neden async?

Veritabanı cevabı beklenirken çalışan thread'i gereksiz yere meşgul etmemek için EF Core'un asenkron metotlarını kullanırız.

Senkron	Asenkron
ToList()	ToListAsync()
FirstOrDefault()	FirstOrDefaultAsync()
SaveChanges()	SaveChangesAsync()

Service örneği

```
public async Task<List<Product>> GetAllAsync()
{
    return await _context.Products.ToListAsync();
}
```

Controller örneği

```
[HttpGet]
public async Task<IActionResult> GetAll()
{
    List<Product> products =
        await _service.GetAllAsync();

    return Ok(products);
}
```

Task neyi anlatır?

Task<List<Product>>, işlemin daha sonra tamamlanacağını ve sonunda List<Product> üreteceğini söyler.

Gerekli namespace

```
using Microsoft.EntityFrameworkCore;
```

3. Proje ve interface dönüşümü

Gün 11 kaynaklarını yeni Gun12-AsyncCrudApi projesine taşı; EF Core paketlerini ve migration dosyalarını koru.

Yeni DTO'lar

```
public class UpdateProductDto
{
    public string Name { get; set; } = "";
    public decimal Price { get; set; }
}

public class UpdatePriceDto
{
    public decimal Price { get; set; }
}
```

IProductService sözleşmesi

```
public interface IProductService
{
    Task<List<Product>> GetAllAsync();
    Task<Product?> GetByIdAsync(int id);
    Task<Product> CreateAsync(CreateProductDto dto);
    Task<Product?> UpdateAsync(
        int id,
        UpdateProductDto dto);
    Task<Product?> UpdatePriceAsync(
        int id,
        UpdatePriceDto dto);
    Task<bool> DeleteAsync(int id);
}
```

Async son eki

Asenkron metot adlarının Async ile bitmesi C# ekosisteminde yaygın ve açıklayıcı isimlendirme kuralıdır.

Dönüşler

- Product? Güncellendi veya bulunamadı.
- bool Silindi ya da bulunamadı.
- Task Sonuç asenkron olarak üretilecek.

4. Asenkron okuma ve oluşturma

ProductService içindeki senkron çağrılar EF Core'un async karşılıklarıyla değiştir.

```
public async Task<List<Product>> GetAllAsync()
{
    return await _context.Products.ToListAsync();
}

public async Task<Product?> GetByIdAsync(int id)
{
    return await _context.Products.FirstOrDefaultAsync(
        product => product.Id == id);
}

public async Task<Product> CreateAsync(
    CreateProductDto dto)
{
    Product product = new Product
    {
        Name = dto.Name,
        Price = dto.Price
    };

    _context.Products.Add(product);
    await _context.SaveChangesAsync();

    return product;
}
```

Add neden async değil?

Add yalnızca EF Core'un takip alanına nesne ekler. Asıl veritabanı I/O işlemi SaveChangesAsync sırasında gerçekleşir.

Controller çağrı kalıbı

```
Product? product =
    await _service.GetByIdAsync(id);
```

İlk kontrol

- GET liste 200 döndürüyor mu?
- GET tek ürün 200/404 döndürüyor mu?
- POST 201 döndürüyor mu?

5. PUT — tam güncelleme

Service önce ürünü bulur; yoksa null döndürür. Varsa Name ve Price alanlarının ikisini de DTO'daki değerlerle değiştirir.

Service iskeleti

```
public async Task<Product?> UpdateAsync(  
    int id,  
    UpdateProductDto dto)  
{  
    Product? product = await _context.Products  
        .FirstOrDefaultAsync(item => item.Id == id);  
  
    if (product is null)  
    {  
        return null;  
    }  
  
    product.Name = /* DTO adı */;  
    product.Price = /* DTO fiyatı */;  
  
    await _context.SaveChangesAsync();  
    return product;  
}
```

Controller iskeleti

```
[HttpPut("{id:int}")]  
public async Task<IActionResult> Update(  
    int id,  
    UpdateProductDto dto)  
{  
    // Adı ve fiyatı doğrula.  
    Product? product =  
        await _service.UpdateAsync(id, dto);  
  
    if (product is null)  
    {  
        return NotFound(new  
        {  
            Message = "Ürün bulunamadı."  
        });  
    }  
  
    return Ok(product);  
}
```

PUT beklentisi

Bu eğitimde PUT body'sinde Name ve Price birlikte zorunludur. Tek alan değişikliği için PATCH kullanacağız.

6. PATCH ve DELETE

PATCH yalnızca fiyatı günceller; DELETE ise ürünü bulup tablodan kaldırır.

PATCH service iskeleti

```
public async Task<Product?> UpdatePriceAsync(
    int id,
    UpdatePriceDto dto)
{
    Product? product = await _context.Products
        .FirstOrDefaultAsync(item => item.Id == id);

    if (product is null) return null;

    product.Price = /* DTO fiyatı */;
    await _context.SaveChangesAsync();
    return product;
}
```

DELETE service iskeleti

```
public async Task<bool> DeleteAsync(int id)
{
    Product? product = await _context.Products
        .FirstOrDefaultAsync(item => item.Id == id);

    if (product is null) return false;

    _context.Products.Remove(product);
    await _context.SaveChangesAsync();
    return true;
}
```

Controller sonuçları

İşlem	Bulundu	Bulunamadı
PATCH	200 + güncel ürün	404
DELETE	204 No Content	404

204 No Content

Silme başarılıdır; response body gönderilmez. Controller içinde NoContent() kullanılır.

7. Test matrisi ve Git

Endpoint'ler

Metot	Adres	Başarı
GET	/api/products	200
GET	/api/products/1	200 / 404
POST	/api/products	201
PUT	/api/products/1	200 / 404
PATCH	/api/products/1/price	200 / 404
DELETE	/api/products/1	204 / 404

Test sırası

- POST ile ürün oluştur.
- PUT ile ad ve fiyatı birlikte değiştir.
- PATCH ile yalnızca fiyatı değiştir; ad aynı kalsın.
- DELETE ile ürünü sil ve 204 gör.
- Aynı ID'ye GET gönderip 404 doğrula.

Mülakat notları

- PUT tam güncelleme, PATCH kısmi güncellemedir.
- Async EF çağrıları veritabanı beklerken thread'i serbest bırakır.
- SaveChangesAsync değişiklikleri kalıcı olarak yazar.
- 204 başarılı ve gövdesiz cevaptır.

Git

```
git add .
git commit -m "Gün 12 async EF Core ve CRUD tamamlandı"
git push
```

Gün 12 tamamlanma ölçütü

GET, POST, PUT, PATCH ve DELETE SQLite üzerinde çalışıyor; silinen ürün tekrar GET ile 404 dönüyorsa hedef tamamdır.